

Using the ODELAYE3 Primitive in UltraScale+ FPGAs

Table of Contents

1. Where Can ODELAYE3 Be Used?	3
2. ODELAYE3 Delay Line Structure	4
3. DELAY_FORMAT: COUNT and TIME	5
3.1. COUNT Mode	5
3.2. TIME Mode	5
4. Why Is IDELAYCTRL Required?	6
5. ODELAYE3 Primitive Ports	7
6. DELAY_TYPE: FIXED, VARIABLE, and VAR_LOAD	10
6.1. FIXED Mode	10
6.2. VARIABLE Mode	10
6.3. VAR_LOAD Mode	10
7. UPDATE_MODE	11
8. EN_VTC and Dynamic Delay Update	11
9. Why Isn't CNTVALUE Calculated Directly?	13

In FPGA designs, timing problems at the clock-cycle level, such as clock frequency, setup/hold times, or clock domain crossing, are generally dealt with. However, especially in high-speed ADC/DAC, source-synchronous interface, or clock/data signals going outside the FPGA, there may be cases where even a difference of a few hundred **picoseconds** is important.

For example, the PCB traces of the clock and data signals sent from the FPGA to an integrated circuit may not be completely equal. In addition, the integrated circuit on the other side may want to see the data at a certain point of the clock. In such a case, delaying the clock by one clock cycle is often meaningless. What is sometimes needed is simply the ability to add very small amounts of delay such as:

```
200 ps  
400 ps  
650 ps
```

One of the resources that can be used for such operations in AMD/Xilinx UltraScale and UltraScale+ FPGAs is the **ODELAYE3** primitive.

ODELAYE3 allows a programmable delay on the order of picoseconds to be added to a signal that will be sent from inside the FPGA to the outside. In the UltraScale architecture, the primitive contains a programmable delay line of **512 taps** and is designed to be used on the output path. ODELAYE3 can be placed between output sources such as programmable logic or OSERDES and the IOB.

UG571 – UltraScale Architecture SelectIO Resources User Guide document, in which AMD describes UltraScale and UltraScale+ SelectIO resources in detail, can be accessed [from this link](#). This document has been used extensively within the scope of this article.

To see the ports and usage details of the ODELAYE3 primitive, [primitive documentation](#) can also be examined.

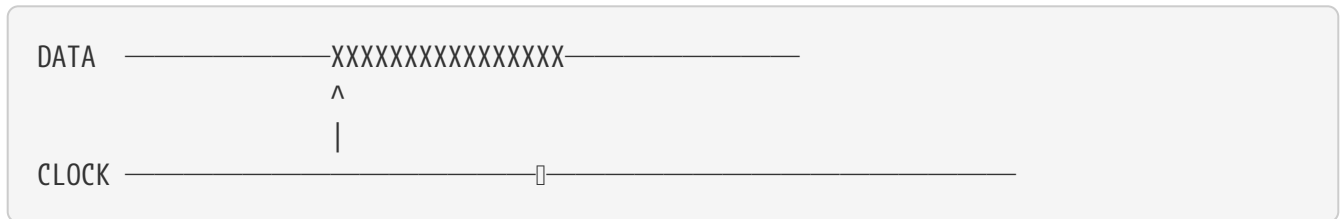
ODELAY is not a general-purpose combinational delay element inside the FPGA. It is a **dedicated** delay resource located on the I/O side. Therefore, it should not be used to randomly delay any RTL signal inside the FPGA.

1. Where Can ODELAYE3 Be Used?

Thinking of the need for ODELAYE3 only as **delaying the signal** would be somewhat superficial. The main purpose is often to be able to fine-tune the **timing relationship** between two signals.

For example, both data and clock may be sent from the FPGA to an ADC or DAC. As a result of PCB propagation delay, FPGA output delay, and the setup/hold requirements of the device on the other side, it may be observed that the data is not at the desired position relative to the clock edge.

Theoretically, the situation may be as follows:



If the data transition point is too close to the sampling edge, the setup or hold margin will decrease.

By adding a small amount of delay to the clock or data path, it may be possible to move the sampling point to a more suitable region within the data eye.

Another area of use is compensating for propagation delay differences between traces on the PCB. In high-speed parallel interfaces, differences of even a few hundred ps may be important.

2. ODELAYE3 Delay Line Structure

There are **512 taps** inside ODELAYE3. This is also the main reason why the **CNTVALUEIN** and **CNTVALUEOUT** ports are 9 bits.

A tap represents one delay step within the delay line. Each tap has a corresponding ps value, and this value may vary depending on many factors.

However, there is a very important detail here. The number of picoseconds corresponding to one tap should not be assumed to be constant in every case.

Process, voltage, and temperature (PVT) changes can affect the physical behavior of the delay line. Therefore, there are two different delay approaches for ODELAYE3 in the UltraScale architecture: **COUNT** and **TIME**.

In the [AMD documentation](#), it is clearly stated that the delay line is not calibrated in COUNT mode, whereas calibration and voltage/temperature compensation are applied by IDELAYCTRL in TIME mode. In **TIME** mode, the use of the **IDELAYCTRL** primitive is required for calibration.

In this case, in **COUNT** mode, the delay value may change over time depending on PVT factor, but this is not the case in **TIME** mode.

3. DELAY_FORMAT: COUNT and TIME

One of the important attributes in ODELAYE3 is **DELAY_FORMAT**.

There are two different options:

DELAY_FORMAT	Meaning from User's Perspective	PVT Compensation
COUNT	Delay is considered as the number of taps	None
TIME	Delay is considered as physical time, that is, ps	Available

3.1. COUNT Mode

When COUNT mode is used, the delay line is controlled directly through the number of taps.

For example, when **CNTVALUEIN** = **100** is specified, 100 taps are selected. Here, a physical duration such as **100 taps** = **250 ps** or **100 taps** = **270 ps** is not guaranteed.

AMD also states that actual tap delay is not important for COUNT mode and that the design should operate only based on the number of taps. In addition, there is no need to use **IDELAYCTRL** in COUNT mode, and **EN_VTC** should be kept Low.

This mode can especially be used in cases where how many taps are moved within a data eye is more important than how many ps the physical delay is.

3.2. TIME Mode

The purpose is different in TIME mode. Here, the delay is considered in terms of physical time, for example 500 ps.

For a single ODELAYE3 in UltraScale+ devices, TIME mode **DELAY_VALUE** range is limited depending on device family. In the UG571 document, an upper limit of 1250 ps is specified for UltraScale devices, while an upper limit of **1100 ps** is specified for UltraScale+ devices.

IDELAYCTRL must be used in TIME mode. The main reason for this is that the delay line is calibrated against voltage and temperature changes.

4. Why Is IDELAYCTRL Required?

One of the most important elements of using TIME mode is the **IDELAYCTRL** primitive.

basic ports in IDELAYCTRL are:

Port	Direction	Description
REFCLK	Input	Reference clock used for delay calibration. A clock in the range of 300 MHz–800 MHz must be provided.
RST	Input	IDELAYCTRL active-high reset.
RDY	Output	Indicates that calibration process has been completed.

IDELAYCTRL uses the reference clock provided to it to calibrate TIME-mode IDELAYE3 and ODELAYE3 delay lines located in the same region against PVT changes.

In the ODELAY primitive that will be described later in this article, the reference clock is used as **333.333333 MHz**.

An example port map for the IDELAYCTRL primitive is given below:

```
-- IDELAYCTRL: IDELAYE3/ODELAYE3 Tap Delay Value Control
IDELAYCTRL_inst : IDELAYCTRL
generic map (
    SIM_DEVICE => "ULTRASCALE"
)
port map (
    RDY    => ctrl_rdy,
    REFCLK => ctrl_refclk,
    RST    => ctrl_rst
);
```

5. ODELAYE3 Primitive Ports

basic ports of the ODELAYE3 primitive are as follows:

Port	Direction	Bit	Description
ODATAIN	Input	1	signal to be delayed.
DATAOUT	Output	1	signal to which ODELAY has been applied.
CLK	Input	1	clock for ODELAY dynamic control operations. It does not have to be the clock of the delayed signal.
CNTVALUEIN	Input	9	new tap value to be loaded when using VAR_LOAD.
CNTVALUEOUT	Output	9	Shows current tap value of the delay line. This value may change over time with PVT changes.
LOAD	Input	1	Allows CNTVALUEIN value to be loaded into the delay line when using VAR_LOAD.
CE	Input	1	Enables increment/decrement operations of the tap count in VARIABLE or VAR_LOAD mode.
INC	Input	1	Used together with CE to determine whether the tap value is increased or decreased.
EN_VTC	Input	1	Controls voltage/temperature compensation mechanism.
RST	Input	1	Resets the delay line to the DELAY_VALUE attribute value.

CASC_IN	Input	1	Used for cascade delay structure.
CASC_OUT	Output	1	Connection to next delay element in cascade structure.
CASC_RETURN	Input	1	Return connection of cascade structure.

CLK port in particular is prone to misunderstanding. The CLK port is not directly the clock of the delayed signal. **CLK** is the clock of control signals such as **LOAD**, **CE**, and **INC**.

An example port map is given below:

```
-- ODELAYE3: Output Fixed or Variable Delay Element
ODELAYE3_inst : ODELAYE3
  generic map (
    CASCADE           => "NONE",
    DELAY_FORMAT      => "TIME",
    DELAY_TYPE        => "VAR_LOAD",
    DELAY_VALUE       => 1000,
    IS_CLK_INVERTED   => '0',
    IS_RST_INVERTED   => '0',
    REFCLK_FREQUENCY  => 333.3333333,
    SIM_DEVICE        => "ULTRASCALE_PLUS",
    UPDATE_MODE       => "ASYNC"
  )
  port map (
    CASC_OUT    => open,
    CNTVALUEOUT => od_cntvalue_out,
    DATAOUT    => od_data_out,
    CASC_IN     => '0',
    CASC_RETURN => '0',
    CE          => '0',
    CLK         => od_clk,
    CNTVALUEIN  => od_cntvalue_in,
    EN_VTC     => od_en_vtc,
    INC         => '0',
    LOAD       => od_load,
    ODATAIN    => od_data_in,
    RST        => od_rst
  );
```

testbench image resulting from the example usage is shown below. ODELAY primitive has a default delay value of 144 ps. This value may be different in real-time tests, and it is thought to probably be primitive propagation delay. In addition, 1000 ps value added as **DELAY_VALUE** can also be observed in the time difference between the signals.

6. DELAY_TYPE: FIXED, VARIABLE, and VAR_LOAD

The second important selection in ODELAYE3 is the **DELAY_TYPE** attribute.

There are three different options:

DELAY_TYPE	Control Method	Advantage	Disadvantage
FIXED	Fixed DELAY_VALUE	Simple and requires little control logic.	delay line cannot be changed at runtime.
VARIABLE	CE + INC	It is easy to move tap by tap.	Large tap changes require many operations and latency.
VAR_LOAD	CNTVALUEIN + LOAD	target tap can be loaded directly.	update sequence must be managed carefully. There are certain rules.

6.1. FIXED Mode

In **FIXED** mode, delay is determined during design. For example:

```
DELAY_TYPE => "FIXED",  
DELAY_VALUE => 500
```

If continuously different delay values will not be tried at runtime, it is a very simple solution.

6.2. VARIABLE Mode

In **VARIABLE** mode, the delay line can be moved tap by tap with the help of **CE** and **INC** signals.

The advantage of this is that it is quite simple. On the other hand, when it is desired to go from 100 taps to 300 taps, many update operations are required.

6.3. VAR_LOAD Mode

In this mode, new tap value is basically applied to **CNTVALUEIN** bus, and the delay line is updated using **LOAD**.

AMD documentation states that **VAR_LOAD** can be used in both **TIME** and **COUNT** formats.

7. UPDATE_MODE

ODELAYE3 also has an **UPDATE_MODE** attribute. There are **ASYNC**, **SYNC**, and **MANUAL** options.

ASYNC has been preferred in IP introduced within the scope of this document.

In **ASYNC** mode, delay update operations are carried out through the ODELAY control clock independently of data input transitions. It should be kept in mind that in **SYNC** mode, unexpected glitch behavior may occur depending on application because the update is associated with input data transition.

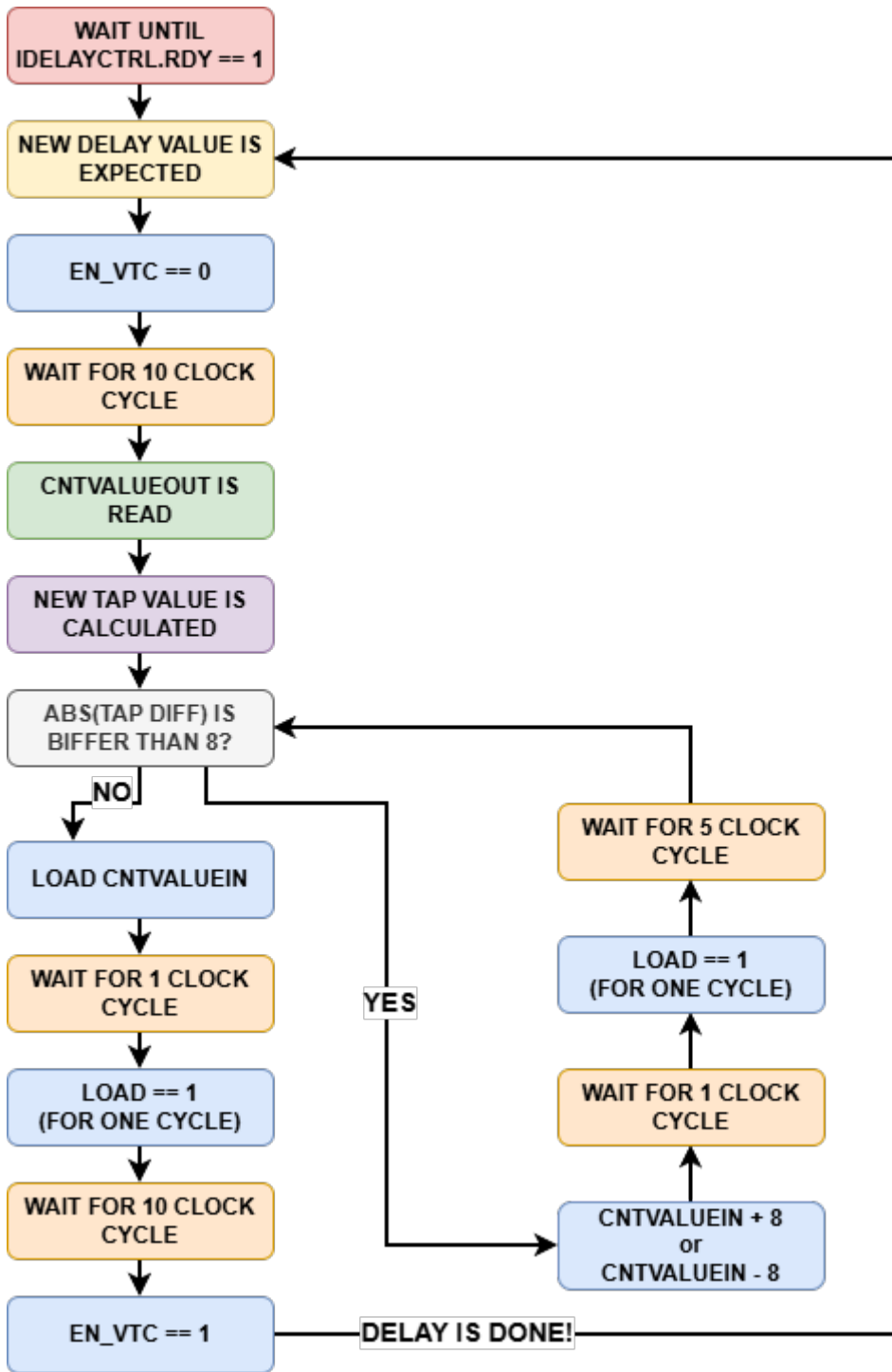
8. EN_VTC and Dynamic Delay Update

One of the most critical signals when using TIME mode is the **EN_VTC** signal.

During normal operation, voltage/temperature compensation can be kept active by keeping **EN_VTC High**.

However, when ODELAY delay value is to be changed, compensation should be temporarily disabled.

Based on steps shared by AMD in VAR_LOAD/TIME procedure, the following flow has been created:



AMD also strongly recommends that large **CNTVALUE** changes be **divided into steps of 8 taps or less**; it states that sudden changes greater than 8 taps can cause glitches and disrupt serialization on the output side. In other words, AMD very strongly recommends reaching the target tap value step by step with a maximum of 8 taps.

9. Why Isn't CNTVALUE Calculated Directly?

One of the interesting aspects of TIME mode appears here. When the user says **I want 500 ps**, 500 cannot be written directly to the runtime interface of ODELAYE3. This is because **CNTVALUEIN**, even in TIME mode, is **in terms of taps**. Although the **DELAY_VALUE** attribute is in terms of ps, the runtime **CNTVALUEIN/CNTVALUEOUT** interface operates with the number of taps.

Therefore, the controller needs to establish a relationship between ps and tap.

For example, in the known case:

```
Current Delay = 1000 ps
Current CNTVALUEOUT = 400
```

let it be.

new request:

```
New Delay = 750 ps
```

can then be approximately calculated as:

```
New CNTVALUE = 400 x 750 / 1000 = 300
```

general expression is:

$$\text{New_CNT} = \text{Current_CNT} \times \frac{\text{Requested_Delay}}{\text{Current_Delay}}$$

It is useful to perform this calculation for every change because the delay value corresponding to each tap may vary according to PVT.